KAZAKH-BRITISH TECHNICAL UNIVERSITY

KBTU

Data collection and preparation

Student Independent Study (SIS) 1

Relationship between Caffeine Intake and Study Effectiveness

Team members:Kumar Aray, Izimbetova AIgerim,
Yerenkyzy Saniya
ID: 23B031179, 23B031098,
23BD031085

Almaty, 2026

Topic. Relationship between Caffeine Intake and Study Effectiveness

1. Objective

The purpose of this project was to apply the complete data collection and preparation pipeline: collecting real data from two different sources (one API and one static web page), cleaning, merging and transforming the datasets, performing exploratory data analysis (EDA), and communicating the insights with visualizations.

The research question was: do countries with a higher caffeine intake per person show higher learning outcomes?

2. Data sources

Individual-level data about caffeine intake and studying is not openly available, so a country-level proxy was used. The complete code, outputs, and plots are included in the attached Jupyter Notebook.

Concept	Variable	Source	Method
Caffeine intake	Caffeine per person per day, mg (coffee, tea, maté, cocoa, all sources); 42 countries, data from 1995	CaffeineInformer web page "Caffeine (Coffee) Consumption By Country" (static HTML table)	requests + BeautifulSoup
Study effectiveness	Harmonized test scores; learning-adjusted years of school (latest value, 2020)	World Bank Indicators API	requests (JSON)
Control variable	GDP per capita, current USD	World Bank Indicators API	requests (JSON)

Tools: Python, pandas, NumPy, Matplotlib, Seaborn, SciPy, requests, BeautifulSoup.

3. Task 1 — Collecting data from the API

The World Bank Indicators API was used (it is open and needs no key). First, the list of countries with their ISO code, region and income level was downloaded; after removing aggregates such as "World", 217 countries remained.

Then three indicators were requested: harmonized test scores (HD.HCI.HLOS), learning-adjusted years of school (HD.HCI.LAYS) and GDP per capita (NY.GDP.PCAP.CD). For each indicator the most recent non-empty value was requested (mrnev=1) together with its year. Learning outcomes are available for 172 countries. All indicators were combined into one table by ISO code and the raw data was saved to CSV.

```python
frames = []
for code_, name in INDICATORS.items():
    payload = wb_get(f"country/all/indicator/{code_}", {"per_page": 20000, "mrnev": 1})
    records = payload[1] or []
    df_ind = pd.DataFrame([
        {"iso3": r["countryiso3code"], name: r["value"], f"{name}_year": r["date"]}
        for r in records
    ])
    df_ind = df_ind[df_ind["iso3"] != ""].drop_duplicates("iso3")
    print(f"{code_:<16} -> {name:<24} {len(df_ind)} records")
    frames.append(df_ind)

wb_values = reduce(lambda a, b: a.merge(b, on="iso3", how="outer"), frames)
wb_raw = countries.merge(wb_values, on="iso3", how="left")
wb_raw.to_csv("data/worldbank_raw.csv", index=False)
wb_raw.head()
```

✓ executed

```
HD.HCI.HLOS      -> test_score              172 records
HD.HCI.LAYS      -> learning_adjusted_years  172 records
NY.GDP.PCAP.CD   -> gdp_per_capita           257 records
```

	iso3	iso2	country	region	income_level	test_score	test_score_year
0	ABW	AW	Aruba	Latin America & Caribbean	High income	NaN	NaN
1	AFG	AF	Afghanistan	Middle East, North Africa, Afghanistan & Pakistan	Low income	354.758789	2020
2	AGO	AO	Angola	Sub-Saharan Africa	Lower middle income	325.965485	2020
3	ALB	AL	Albania	Europe & Central Asia	Upper middle income	434.127594	2020
4	AND	AD	Andorra	Europe & Central Asia	High income	NaN	NaN

	learning_adjusted_years	learning_adjusted_years_year	gdp_per_capita	gdp_per_capita_year
0	NaN	NaN	38590.565029	2024
1	5.052838	2020	416.871146	2024
2	4.234978	2020	3129.476623	2025
3	8.953018	2020	12998.147870	2025
4	NaN	NaN	54291.502603	2025

Figure 1. Downloading the three indicators and building the raw World Bank table.

4. Task 2 — Web scraping

Before scraping, robots.txt of the website was checked: the request returned status 200 and scraping of the page is allowed. The page was downloaded with requests (HTTP 200) and parsed with BeautifulSoup. One HTML table was found (44 rows, with two header rows).

A parsing function selects the widest row that starts with "Country" as the header, keeps only the rows that match the header length, and the columns were renamed. The result contains 42 countries with caffeine intake from coffee, tea, maté, cocoa and all sources. The raw scraped data was saved to CSV.

```
robots_resp = requests.get("https://www.caffeineinformer.com/robots.txt", headers=HEADERS, timeout=30)
print("robots.txt status:", robots_resp.status_code)

rp = urllib.robotparser.RobotFileParser()
rp.parse(robots_resp.text.splitlines() if robots_resp.status_code == 200 else [])

print("Scraping allowed:", rp.can_fetch(HEADERS["User-Agent"], CAFFEINE_URL))
```

✓ executed

```
robots.txt status: 200
Scraping allowed: True
```

Figure 2. robots.txt check.

```
response = requests.get(CAFFEINE_URL, headers=HEADERS, timeout=30)
response.raise_for_status()
soup = BeautifulSoup(response.text, "html.parser")

tables = soup.find_all("table")
print("HTTP status:", response.status_code)
print("Tables found:", len(tables))

for i, t in enumerate(tables):
    first_rows = [[c.get_text(" ", strip=True) for c in tr.find_all(["th", "td"])] for tr in t.find_all("tr")[:3]]
    print(i, "| rows:", len(t.find_all("tr")), "| first rows:", first_rows)
```

✓ executed

```
HTTP status: 200
Tables found: 1
0 | rows: 44 | first rows: [['', 'Caffeine Source (mg / day)'], ['Country', 'Coffee', 'Tea', 'Maté', 'Cocoa', 'All Sources'], ['Algeria', '79', '5', '0', '1', '85']]
```

Figure 3. Downloading the page and inspecting the table structure.

```python
candidates = [df_ for df_ in (parse_caffeine_table(t) for t in tables) if df_ is not None]
if not candidates:
    raise RuntimeError("No table with a 'Country' header found - inspect the output of the previous cell.")

caffeine_raw = max(candidates, key=len)


def ascii_lower(text):
    return unicodedata.normalize("NFKD", text).encode("ascii", "ignore").decode().lower().strip()


COLUMN_MAP = {"country": "country_raw", "coffee": "coffee_mg", "tea": "tea_mg", "mate": "mate_mg",
              "cocoa": "cocoa_mg", "all sources": "caffeine_mg_per_day"}
caffeine_raw.columns = [COLUMN_MAP.get(ascii_lower(c), ascii_lower(c)) for c in caffeine_raw.columns]
caffeine_raw.to_csv("data/caffeine_raw.csv", index=False)

print("Scraped rows:", len(caffeine_raw))
print("Columns:", list(caffeine_raw.columns))
caffeine_raw.head(10)
```

✓ executed

```
Scraped rows: 42
Columns: ['country_raw', 'coffee_mg', 'tea_mg', 'mate_mg', 'cocoa_mg', 'caffeine_mg_per_day']
```

	country_raw	coffee_mg	tea_mg	mate_mg	cocoa_mg	caffeine_mg_per_day
0	Algeria	79	5	0	1	85
1	Angola	4	–	0	0	4
2	Argentina	43	1	52	5	100
3	Australia	202	29	0	0	232
4	Austria	276	8	0	16	300
5	Brazil	26	1	10	4	40
6	Canada	180	18	0	12	210
7	China	2	14	0	0	16
8	Colombia	126	0	0	9	136
9	Denmark	354	15	0	21	390

Figure 4. Parsed table: 42 rows and 6 columns.

5. Task 3 — Data cleaning

First, both raw tables were inspected. All scraped values were stored as text, and a dash ("–") was used for missing values. In the World Bank table, 45 countries have no learning-outcome data and 3 countries have no GDP value.

The scraped table was cleaned by converting all columns to numeric types (dashes became missing values), dropping rows without the all-sources value, removing duplicates and aggregate rows, and checking that values are realistic (0–1000 mg/day). No rows had to be removed (42 rows before and after); only the tea column contains missing values (3).

The World Bank table was cleaned by converting indicators to numeric types, years to integers, removing duplicates, and dropping countries without any learning-outcome data; 172 countries

remained. Finally, country names were standardized (lowercase, no accents or punctuation) and a small dictionary of aliases was added, because the two sources spell some names differently.

```
caffeine = caffeine_raw.copy()
caffeine["country_raw"] = caffeine["country_raw"].map(clean_text)

value_cols = [c for c in ["coffee_mg", "tea_mg", "mate_mg", "cocoa_mg", "caffeine_mg_per_day"] if c in caffeine.columns]
for c in value_cols:
    caffeine[c] = pd.to_numeric(caffeine[c].astype(str).str.replace(",", "", regex=False), errors="coerce")  # "-" -> NaN

n0 = len(caffeine)
caffeine = caffeine.dropna(subset=["caffeine_mg_per_day"])                                  # missing values
caffeine = caffeine[~caffeine["country_raw"].str.contains(r"^(?:world|total|average)", case=False, regex=True)]
caffeine = caffeine.drop_duplicates(subset="country_raw", keep="first")                     # duplicates
caffeine = caffeine[caffeine["caffeine_mg_per_day"].between(0, 1000)]                       # sanity check (mg/day)

print(f"Rows: {n0} -> {len(caffeine)} after cleaning")
caffeine[value_cols].describe()

✓   executed
```

```
Rows: 42 -> 42 after cleaning
```

	coffee_mg	tea_mg	mate_mg	cocoa_mg	caffeine_mg_per_day
count	42.000000	39.000000	42.000000	42.000000	42.000000
mean	125.642857	25.666667	4.000000	5.857143	159.428571
std	121.275703	32.189679	17.361297	5.524100	120.262141
min	1.000000	0.000000	0.000000	0.000000	4.000000
25%	23.750000	3.500000	0.000000	1.000000	52.000000
50%	86.500000	12.000000	0.000000	4.500000	153.000000
75%	201.000000	35.500000	0.000000	9.000000	219.000000
max	388.000000	127.000000	101.000000	21.000000	414.000000

Figure 5. Cleaning of the scraped table.

```python
wb = wb_raw.copy()

num_cols = ["test_score", "learning_adjusted_years", "gdp_per_capita"]
for c in num_cols:
    wb[c] = pd.to_numeric(wb[c], errors="coerce")
for c in [f"{n}_year" for n in num_cols]:
    wb[c] = pd.to_numeric(wb[c], errors="coerce").astype("Int64")

wb = wb.drop_duplicates(subset="iso3")
wb = wb.dropna(subset=["test_score", "learning_adjusted_years"], how="all")   # no outcome data -> useless

print("Countries with at least one learning-outcome value:", len(wb))
wb[num_cols].describe().round(2)
```

✓ executed

```
Countries with at least one learning-outcome value: 172
```

	test_score	learning_adjusted_years	gdp_per_capita
count	172.00	172.00	172.00
mean	423.83	7.82	19918.35
std	63.23	2.48	26802.15
min	304.92	2.21	233.82
25%	378.37	6.00	2700.41
50%	414.32	7.94	8029.74
75%	474.17	9.76	26986.98
max	575.27	12.81	147252.18

Figure 6. Cleaning of the World Bank table.

6. Task 4 — Merging

The two datasets were merged with an inner join on the normalized country name (validated as one-to-one). Three countries from the scraped table (Russian Fed., Syrian Arab Republic and Venezuela) had no match in the World Bank data, so 39 countries remained. A new feature, log_gdp = log10(GDP per capita), was created because GDP is strongly right-skewed.

The final dataset has 39 rows and 12 columns without missing values and was saved to CSV.

```
only_caffeine = sorted(set(caffeine["key"]) - set(wb["key"]))
only_wb = sorted(set(wb["key"]) - set(caffeine["key"]))

print(f"In caffeine table but not in World Bank data ({len(only_caffeine)}):")
for k in only_caffeine:
    suggestion = difflib.get_close_matches(k, wb["key"].tolist(), n=1, cutoff=0.8)
    print(f"  {k:<35} -> suggestion: {suggestion}")

print(f"\nIn World Bank data but not in caffeine table: {len(only_wb)} countries (expected, the caffeine list is shorter)")
```

✓ executed

```
In caffeine table but not in World Bank data (3):
  russian fed                         -> suggestion: []
  syrian arab republic                -> suggestion: []
  venezuela rb                        -> suggestion: []

In World Bank data but not in caffeine table: 133 countries (expected, the caffeine list is shorter)
```

Figure 7. Countries that could not be matched between the sources.

```
merged = caffeine[["key", "country_raw", "caffeine_mg_per_day"]].merge(wb, on="key", how="inner", validate="one_to_one")

merged = merged.rename(columns={"country": "country_wb"})
merged["country"] = merged["country_wb"]
merged["log_gdp"] = np.log10(merged["gdp_per_capita"])

final_cols = ["country", "iso3", "region", "income_level", "caffeine_mg_per_day", "test_score", "test_score_year",
              "learning_adjusted_years", "learning_adjusted_years_year", "gdp_per_capita", "gdp_per_capita_year", "log_gdp"]
df = merged[final_cols].sort_values("caffeine_mg_per_day", ascending=False).reset_index(drop=True)

print(f"Merged dataset: {df.shape[0]} countries, {df.shape[1]} columns")
df.head(10)
```

✓ executed

```
Merged dataset: 39 countries, 12 columns
```

	country	iso3	region	income_level	caffeine_mg_per_day	test_score	test_score_year	learning_adjusted_years	\
0	Netherlands	NLD	Europe & Central Asia	High income	414	519.634644	2020	11.539022	
1	Sweden	SWE	Europe & Central Asia	High income	407	519.361755	2020	11.580874	
2	Norway	NOR	Europe & Central Asia	High income	400	513.587830	2020	11.233873	
3	Denmark	DNK	Europe & Central Asia	High income	390	517.877808	2020	11.098828	
4	Finland	FIN	Europe & Central Asia	High income	329	533.707642	2020	11.738752	
5	Germany	DEU	Europe & Central Asia	High income	313	517.281433	2020	11.033090	
6	Austria	AUT	Europe & Central Asia	High income	300	507.639374	2020	10.906205	
7	Switzerland	CHE	Europe & Central Asia	High income	288	515.139038	2020	10.933773	
8	France	FRA	Europe & Central Asia	High income	239	510.260590	2020	11.268723	
9	Australia	AUS	East Asia & Pacific	High income	232	515.685364	2020	11.222505	

	learning_adjusted_years_year	gdp_per_capita	gdp_per_capita_year	log_gdp
0	2020	73683.917133	2025	4.867373
1	2020	63133.212674	2025	4.800258
2	2020	94594.192957	2025	4.975864
3	2020	76970.153522	2025	4.886322
4	2020	56148.580949	2025	4.749339
5	2020	60496.435082	2025	4.781730
6	2020	62930.035132	2025	4.798858
7	2020	114769.012314	2025	5.059825
8	2020	48985.730781	2025	4.690070
9	2020	65129.722799	2025	4.813779

Figure 8. Merged dataset (first rows).

7. Task 5 — Exploratory data analysis

The average caffeine intake in the merged sample is 163.5 mg per day (from 4 to 414 mg), the average test score is 450.7 (from 309 to 538), and the average GDP per capita is about 36,245 USD. GDP is the most skewed variable (skewness 0.93), which justifies the use of its logarithm. No outliers were found by the IQR rule, so all countries were kept.

The highest caffeine intake is observed in the Netherlands (414 mg), Sweden (407), Norway (400), Denmark (390) and Finland (329). The lowest intake is in Angola and Nigeria (4 mg), Tanzania (7), China (16) and Côte d'Ivoire (20).

```
num_vars = ["caffeine_mg_per_day", "test_score", "learning_adjusted_years", "gdp_per_capita"]
df[num_vars].describe().round(2)

✓  executed
```

	caffeine_mg_per_day	test_score	learning_adjusted_years	gdp_per_capita
count	39.00	39.00	39.00	39.00
mean	163.51	450.71	8.96	36244.80
std	123.72	69.05	2.43	34757.82
min	4.00	309.02	4.23	1224.25
25%	45.00	395.48	7.03	6324.36
50%	162.00	448.04	9.27	28419.58
75%	226.50	516.48	11.23	59049.20
max	414.00	537.72	11.74	131592.46

Figure 9. Descriptive statistics.

Correlation analysis shows a strong positive relationship between caffeine intake and test scores (Pearson $r = 0.745$, Spearman rho = 0.718, $p < 0.001$). Similar values were obtained for learning-adjusted years of school ($r = 0.756$) and for GDP per capita ($r = 0.745$), which means that wealth is strongly related to both variables.

```
def corr_table(data, x, ys):
    rows = []
    for y in ys:
        sub = data[[x, y]].dropna()
        r, p = stats.pearsonr(sub[x], sub[y])
        rho, ps = stats.spearmanr(sub[x], sub[y])
        rows.append({"variable": y, "n": len(sub), "pearson_r": r, "pearson_p": p,
                     "spearman_rho": rho, "spearman_p": ps})
    return pd.DataFrame(rows).round(3)


corr_res = corr_table(df, "caffeine_mg_per_day", ["test_score", "learning_adjusted_years", "gdp_per_capita"])
corr_res

✓  executed
```

	variable	n	pearson_r	pearson_p	spearman_rho	spearman_p
0	test_score	39	0.745	0.0	0.718	0.0
1	learning_adjusted_years	39	0.756	0.0	0.765	0.0
2	gdp_per_capita	39	0.745	0.0	0.818	0.0

Figure 10. Pearson and Spearman correlations of caffeine intake.

To check the role of wealth, a partial correlation was calculated (correlation of regression residuals after removing the effect of log GDP). It fell from r = 0.745 to r = 0.306 (p = 0.058), i.e. it is less than half of the raw value and is no longer statistically significant at the 5% level.

```
def residuals(y, x):
    slope, intercept, *_ = stats.linregress(x, y)
    return y - (intercept + slope * x)


sub = df[["caffeine_mg_per_day", "test_score", "log_gdp"]].dropna()
res_caffeine = residuals(sub["caffeine_mg_per_day"], sub["log_gdp"])
res_score = residuals(sub["test_score"], sub["log_gdp"])

r_raw, p_raw = stats.pearsonr(sub["caffeine_mg_per_day"], sub["test_score"])
r_part, p_part = stats.pearsonr(res_caffeine, res_score)

print(f"n = {len(sub)}")
print(f"Raw correlation (caffeine vs test score):        r = {r_raw:.3f}, p = {p_raw:.4f}")
print(f"Partial correlation (controlling for log GDP): r = {r_part:.3f}, p = {p_part:.4f}")

✓  executed
```

```
n = 39
Raw correlation (caffeine vs test score):        r = 0.745, p = 0.0000
Partial correlation (controlling for log GDP): r = 0.306, p = 0.0584
```

Figure 11. Raw and partial correlation.

Finally, the countries were divided into three equal groups by caffeine intake. The average test score grows from 388.7 (low) to 460.0 (medium) and 507.0 (high), but the average GDP per capita grows in parallel from about 7,870 to 70,970 USD.

```
df["caffeine_tier"] = pd.qcut(df["caffeine_mg_per_day"], q=3, labels=["Low", "Medium", "High"])

tier_summary = df.groupby("caffeine_tier", observed=True).agg(
    countries=("country", "count"),
    mean_caffeine_mg=("caffeine_mg_per_day", "mean"),
    mean_test_score=("test_score", "mean"),
    median_test_score=("test_score", "median"),
    mean_gdp=("gdp_per_capita", "mean"),
).round(1)
tier_summary

✓  executed
```

caffeine_tier	countries	mean_caffeine_mg	mean_test_score	median_test_score	mean_gdp
Low	13	30.9	388.7	399.0	7869.7
Medium	14	159.2	460.0	470.5	32831.0
High	12	312.2	507.0	516.5	70967.2

Figure 12. Comparison of caffeine-intake tiers.

8. Task 6 — Visualization

Five labeled plots were created with Matplotlib and Seaborn.

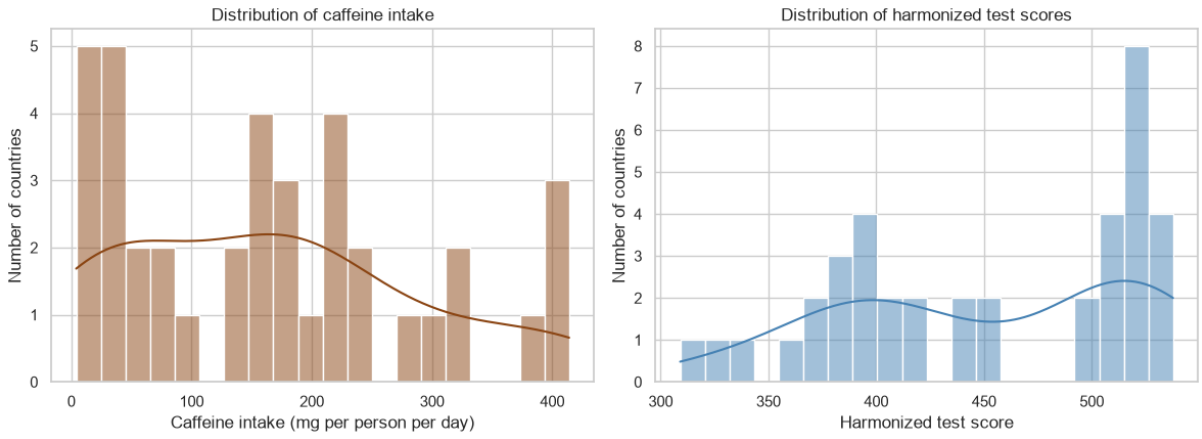


Figure 13. Plot 1. Distributions of caffeine intake and test scores.

Caffeine intake is spread widely, with many countries below 100 mg and a group above 300 mg per day. Test scores form two clusters: around 390–400 and around 515.

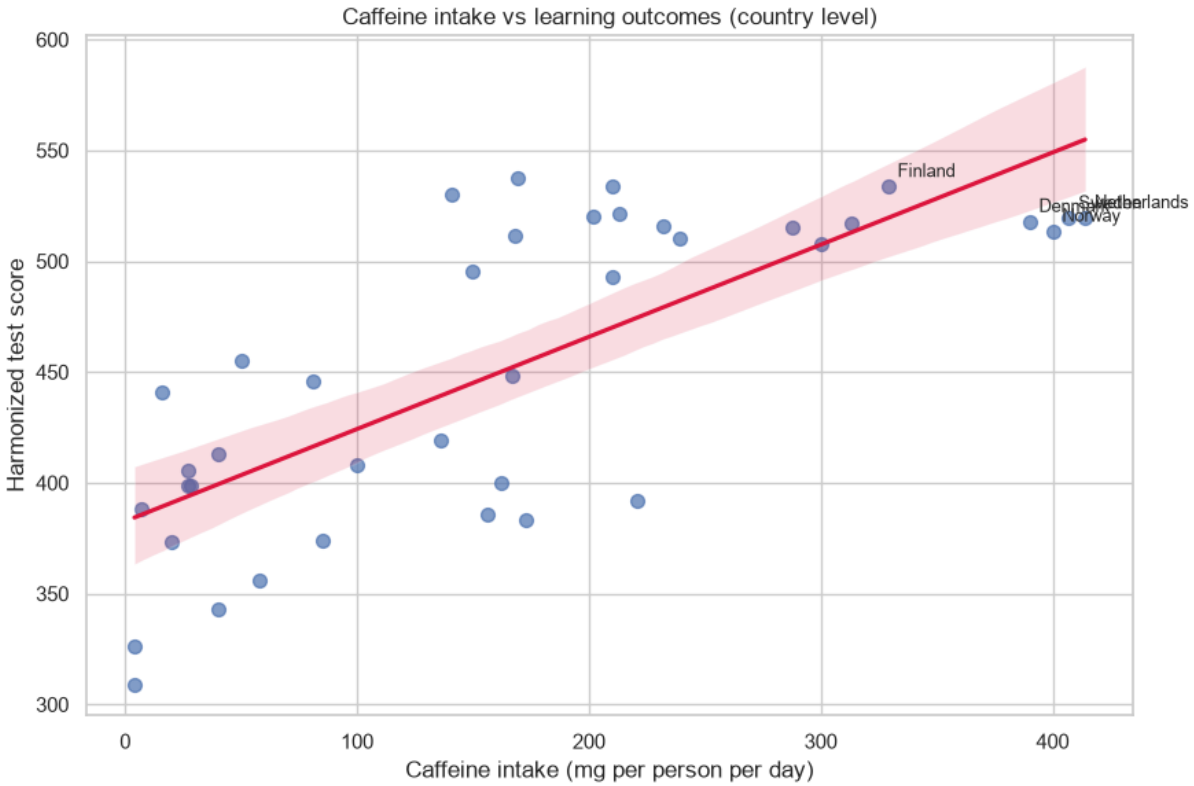


Figure 14. Plot 2. Caffeine intake vs test score with a regression line.

The regression line is clearly upward: higher caffeine intake goes together with higher test scores. The top-right corner is occupied by the Nordic countries and the Netherlands.

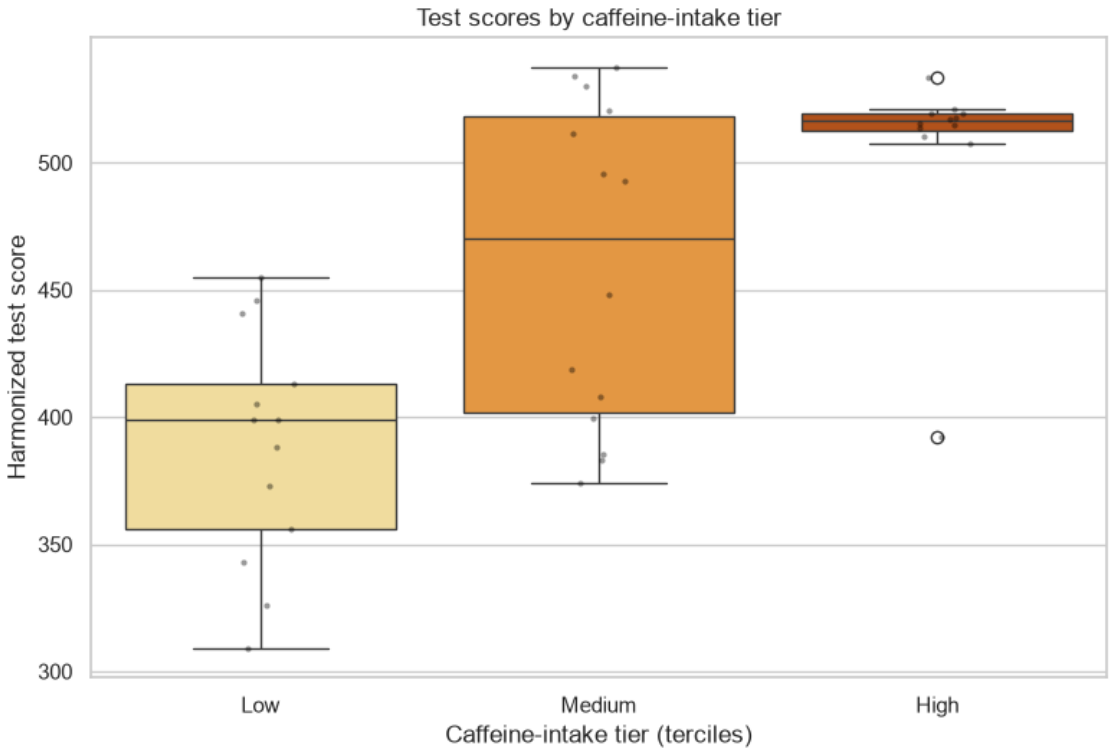


Figure 15. Plot 3. Test scores by caffeine-intake tier.

The median test score rises from the low tier (399) to the medium (470.5) and high (516.5) tiers.

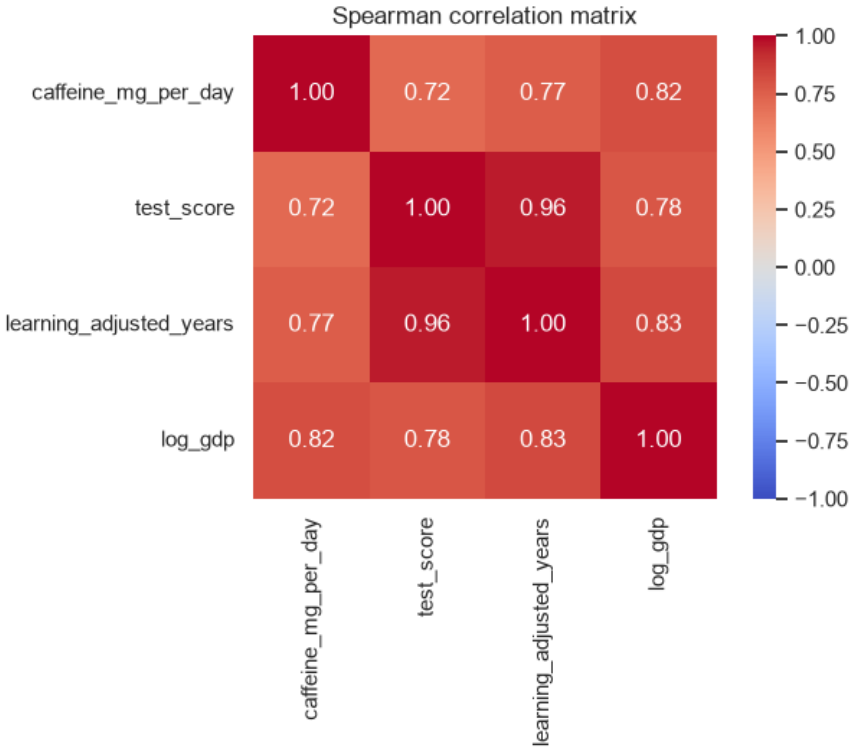


Figure 16. Plot 4. Spearman correlation matrix.

Caffeine intake correlates with test scores (0.72) and with log GDP (0.82), and log GDP also correlates with test scores (0.78), so wealth is related to every other variable in the dataset.

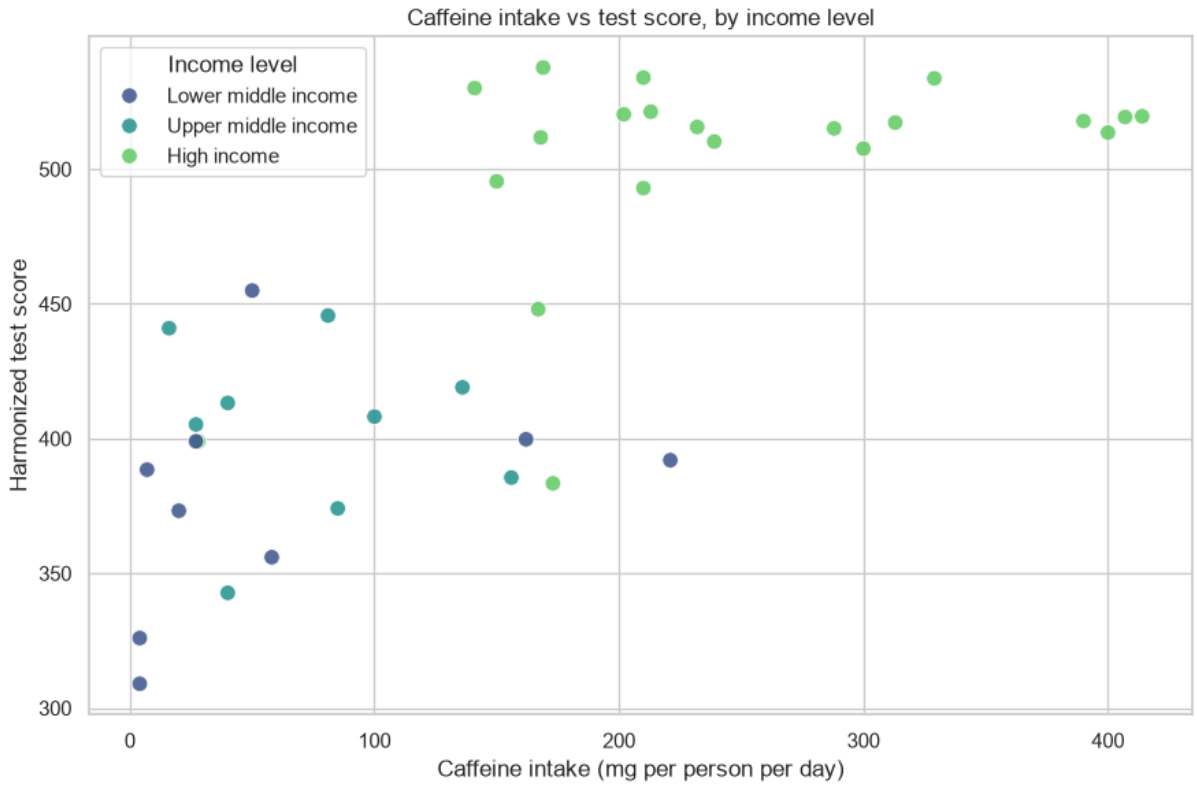


Figure 17. Plot 5. Caffeine intake vs test score, colored by income level.

High-income countries occupy the upper part of the chart, and most of them score above 490 across a wide range of caffeine intake, while lower-middle and upper-middle-income countries are concentrated in the lower left.

9. Key findings

- Countries with higher caffeine intake have clearly higher learning outcomes: r = 0.74 for test scores and r = 0.76 for learning-adjusted years of school.
- Wealth is a strong confounder: after controlling for GDP per capita the correlation falls to r = 0.31 (p = 0.058).
- The tier comparison shows a steady growth of test scores from low to high caffeine intake, but GDP per capita grows together with it.
- The data does not allow us to conclude that caffeine improves study effectiveness.

```
def strength(r):
    a = abs(r)
    return "negligible" if a < 0.1 else "weak" if a < 0.3 else "moderate" if a < 0.5 else "strong"


row = corr_res[corr_res["variable"] == "test_score"].iloc[0]
print(f"Countries analysed: {len(df)} (with test scores: {df['test_score'].notna().sum()})")
print(f"Caffeine vs test score: Pearson r = {row['pearson_r']:.2f} (p = {row['pearson_p']:.3f}), "
      f"Spearman rho = {row['spearman_rho']:.2f} -> {strength(row['pearson_r'])} "
      f"{'positive' if row['pearson_r'] > 0 else 'negative'} association")
print(f"After controlling for GDP: partial r = {r_part:.2f} (p = {p_part:.3f}) -> {strength(r_part)} association")

if abs(r_part) < abs(r_raw) * 0.6:
    print("Interpretation: most of the raw association disappears once wealth is controlled -> GDP is a strong confounder.")
else:
    print("Interpretation: the association stays noticeable after controlling for wealth.")

✓  executed
```

```
Countries analysed: 39 (with test scores: 39)
Caffeine vs test score: Pearson r = 0.74 (p = 0.000), Spearman rho = 0.72 -> strong positive association
After controlling for GDP: partial r = 0.31 (p = 0.058) -> moderate association
Interpretation: most of the raw association disappears once wealth is controlled -> GDP is a strong confounder.
```

Figure 18. Automatic summary of the key results in the notebook.

10. Conclusion

This project demonstrated the full data pipeline: data was collected from an API (World Bank) and from a static web page (CaffeineInformer), cleaned, merged on standardized country names, analyzed with descriptive statistics, correlations and a partial correlation, and visualized with five plots. A strong positive relationship between national caffeine intake and learning outcomes was found ($r = 0.74$), but most of it is explained by the wealth of the countries (partial $r = 0.31$). Individual-level data and more control variables would be needed to study the real effect of caffeine on studying.

The complete code, outputs, and plots are included in the attached Jupyter Notebook.